

Lemma Discovery from Proof Scripts

1 Background

Let $\mathcal{H}$ denote a universe of hypotheses and let $\mathcal{G}$ denote a universe of goals. A *proof obligation* is a pair (Γ, g) where $\Gamma \subseteq \mathcal{H}$ is finite and $g \in \mathcal{G}$. Let

$$\mathcal{O} \triangleq \{ (\Gamma, g) \mid \Gamma \subseteq \mathcal{H}, \Gamma \text{ is finite, } g \in \mathcal{G} \}$$

denote the set of proof obligations.

A *proof tree* is defined inductively by the grammar

$$T ::= a \mid a \langle T_1, \dots, T_m \rangle,$$

where each a is an atomic proof transformation (tactic application) denoting a partial map $\llbracket a \rrbracket : \mathcal{O} \rightarrow \mathcal{P}(\mathcal{O})$. Applying T on a proof obligation $\llbracket T \rrbracket(\Gamma, g)$ yields a finite set of obligations $\{(\Gamma_1, g_1), \dots, (\Gamma_n, g_n)\}$.

To represent only part of a proof tree, we define the *partial proof trees*, generated by

$$\Upsilon ::= \bullet_\ell \mid a \mid a \langle \Upsilon_1, \dots, \Upsilon_m \rangle.$$

Intuitively, a partial proof tree specifies a connected portion of the original proof that is to be replaced by an extracted lemma. The leaf $\bullet_\ell$ marks a boundary point where the selected subtree stops and the original proof continues outside the subtree. Here, ℓ is a hole identifier used to associate each hole with the omitted continuation rooted there. The leaf a denotes a tactic occurrence included in the selected fragment with no further selected descendants. A complete proof tree is therefore a special case of a partial proof tree with no holes.

A *proof* is a tuple $P = ((\Gamma, g), T)$ where $\llbracket T \rrbracket(\Gamma, g) = \emptyset$. Given a proof $P = ((\Gamma, g), T)$, a *proof fragment* of P is a tuple $F_P = ((\Gamma_F, g_F), \Upsilon)$ where Υ is a partial proof tree obtained from a subtree of T rooted at (Γ_F, g_F) by replacing zero or more descendant nodes with holes.

A proof fragment encodes a reusable reasoning pattern that can be internalized as a proposition. Given a fragment $F_P = ((\Gamma_F, g_F), \Upsilon)$, we construct a lemma $\text{Lem}(F_P)$ by recursion on Υ :

$$\begin{aligned} \text{Lem}(\bullet_\ell[(\Gamma_\ell, g_\ell)]) &= g_\ell \\ \text{Lem}(a[(\Gamma, g)]) &= \top \\ \text{Lem}(a[(\Gamma, g)] \langle \Upsilon_1, \dots, \Upsilon_m \rangle) &= \bigwedge_{\substack{i=1 \\ \text{Lem}(\Upsilon_i) \neq \top}}^m \left(\bigwedge_{(\Gamma_i \setminus \Gamma) \rightarrow \text{Lem}(\Upsilon_i)} \right) \end{aligned}$$

where each Υ_i is rooted at obligation (Γ_i, g_i) . When $\Gamma_i \setminus \Gamma = \emptyset$, the implication $(\bigwedge \emptyset) \rightarrow \text{Lem}(\Upsilon_i)$ reduces to $\text{Lem}(\Upsilon_i)$. Then add to the output list the candidate lemma

$$\ell_\Upsilon \triangleq \bigwedge A \wedge \text{Lem}(\Upsilon) \rightarrow g_F.$$

The candidate lemmas extractable from a proof P are thus determined by the fragments of P .

The antecedent of $\text{Lem}(F_P)$ includes the entire input context Γ_F , which typically contains hypotheses the fragment never uses. A useful lemma should retain only the hypotheses required for the fragment to replay. We formalize “replay” as follows.

Definition 1 (Replayable subcontext). *Let $\llbracket \Upsilon \rrbracket(\Gamma, g) = \{(\Gamma_1, g_1), \dots, (\Gamma_m, g_m)\}$. A subset $\Gamma' \subseteq \Gamma$ is replayable for Υ at (Γ, g) if there exist contexts $\Delta_1, \dots, \Delta_m$ such that*

$$\llbracket \Upsilon \rrbracket(\Gamma', g) = \{(\Delta_1, g_1), \dots, (\Delta_m, g_m)\}.$$

That is, the same partial proof tree Υ can be replayed from the smaller input obligation (Γ', g) and produces the same residual goals $g_1, \dots, g_m$, possibly under smaller residual contexts $\Delta_1, \dots, \Delta_m$.

Given a proof fragment $((\Gamma, g), \Upsilon)$, the minimization task is to remove as many assumptions from the input context as possible while preserving the ability to replay Υ and obtain the same residual goals. Formally, let

$$\mathcal{R}_{\Upsilon, \Gamma, g} \triangleq \{ \Gamma' \subseteq \Gamma \mid \Gamma' \text{ is replayable for } \Upsilon \text{ at } (\Gamma, g) \}.$$

Subproblem 1 (Support minimization). The objective is to compute an inclusion-minimal element of $\mathcal{R}_{\Upsilon, \Gamma, g}$. That is, we wish to solve the optimization problem

$$\text{find } \Gamma' \subseteq \Gamma \text{ such that } \Gamma' \in \mathcal{R}_{\Upsilon, \Gamma, g} \text{ and } \forall \Gamma'' \subsetneq \Gamma'. \Gamma'' \notin \mathcal{R}_{\Upsilon, \Gamma, g}.$$

Any such solution Γ' induces the extracted lemma

$$\left(\bigwedge \Gamma' \right) \wedge \text{Lem}(\Upsilon) \rightarrow g.$$

Thus, the problem is not merely to find a sound antecedent, but to find one that is minimal among the subcontexts from which the given proof fragment remains replayable.

Solving Subproblem 1 yields, for each proof fragment, a lemma with inclusion-minimal premises. The quality of the overall lemma discovery procedure therefore depends on the set of fragments considered. Exhaustive fragment exposure requires the notion of proof equivalence.

We will formalize proof equivalence in §2.1 via the proof ordering graph, but intuitively, two proofs are *equivalent* if they share the same set of tactic applications, and agree on input and final output obligations but may differ in their intermediate obligations. Since (Γ_F, g_F) and $\llbracket \Upsilon \rrbracket(\Gamma_F, g_F)$ are intermediate obligations of P , the fragments extractable from P are in general a proper subset of those extractable from the equivalence class of P . Enumerating only the fragments of P therefore misses lemma candidates discoverable from equivalent reorderings of P , motivating the following subproblem.

Subproblem 2 (Fragment Discovery Modulo Proof Equivalence). Given a proof P , identify all possible fragments of proofs equivalent to P .

2 Fragment Discovery Modulo Proof Equivalence

2.1 Proof Ordering Graph

To enumerate proof fragments modulo equivalence, we require a representation of the equivalence class of the input proof: we will define, given a proof T , the proof ordering graph (POG) G_T from which the equivalence class of T can be derived from. The nodes of G_T are the tactic applications of T , and edges record the dependency constraints that any equivalent proof must respect.

Definition 2 (Tactic Footprint). *Let a be a tactic application at obligation (Γ, g) producing residual obligations $(\Gamma_1, g_1), \dots, (\Gamma_m, g_m)$. The footprint of a at (Γ, g) is a tuple $\Phi(a, \Gamma, g) = (D, M, \mu, \rho)$ where:*

1. $D \subseteq \Gamma$ is the set of hypotheses that a uses but does not modify
2. $M \subseteq \{g\} \cup \Gamma$ is the set of propositions that a modifies
3. $\mu : (s : M) \rightarrow \mathcal{P}(\text{Pos}(s))$ maps each modified proposition to the set of positions within it that a modifies
4. ρ is a map tracking how positions propagate forward through the tactic. For each proposition $s \in \{g\} \cup \Gamma$, position $p \in \text{Pos}(s)$, $\rho_i(s, p) = (q, p')$ where $q \in \{g_i\} \cup \Gamma_i$ and $p' \in \text{Pos}(q)$.

Let $a_1, \dots, a_n$ be the sequence of tactic applications along a root-to-leaf path in a proof tree T , with footprints $\Phi(a_k) = (D_k, M_k, \mu_k, \rho_k)$ and input obligations O_k . For indices $i < j$, extend ρ to a forward propagation $\rho^{i \rightarrow j}$ along the path:

$$\rho^{i \rightarrow i}(s, p) = (s, p), \quad \rho^{i \rightarrow j+1}(s, p) = \rho_{j+1}(\rho^{i \rightarrow j}(s, p)).$$

The tactic applications a_j depends on a_i , written $a_i \prec a_j$, if either:

1. (Modification dependence.) $\exists s \in M_i$, $p \in \mu_i(s)$ such that $\exists q \in M_j$, $p' \in \mu_j(q)$ where $(q, p') = \rho^{i \rightarrow j}(s, p)$, and no index k with $i < k < j$ satisfies the same condition relative to a_j ; or
2. (Use dependence.) $\exists _ \in M_i$ such that $\exists q \in D_j$ where $(q, _) = \rho^{i \rightarrow j}(s, _)$, and no index k with $i < k < j$ satisfies the same condition relative to a_j .

Before finally defining the Proof Ordering graph, we need the notion of a branch path. Given a tactic application a in proof tree T , a *branching tactic* is one that outputs more than one residual obligation, and a *branching ancestor* of a is an ancestor tactic application that is branching. The *branch path* $B(a)$ of a is the sequence of branch indices chosen at the branching ancestors of a , with an asterisk suffix $*$ appended if a is itself branching. Tactics lying on a linear chain (no branching ancestors and not themselves branching) share the same branch path.

Definition 3 (Proof Ordering Graph). *Let $N(T)$ denote the set of tactic applications of a proof tree T , defined inductively by*

$$N(a) = \{a\}, \quad N(a \circ \langle T_1, \dots, T_m \rangle) = \{a\} \cup N(T_1) \cup \dots \cup N(T_m).$$

The proof ordering graph of T is the directed graph $G_T = (V, E)$ with $V = N(T)$ and $E = \{(a_i, a_j) \in V \times V \mid a_i \prec a_j\}$. Each node $a \in V$ is annotated with its branch path $B(a)$.

A *linearization* of a POG $G = (V, E)$ is a proof tree T , where $N(T) = V$, and whose root-to-leaf orderings is a topological sort of G that respects its branch ordering.

Denote all linearizations of G as $\mathcal{L}(G)$. For all $T, T' \in \mathcal{L}(G)$, T and T' are equivalent, written $T \equiv T'$.

Theorem 1 (Theorem 1). *If $T \equiv T'$, then $\llbracket T \rrbracket(\Gamma, g) = \llbracket T' \rrbracket(\Gamma, g)$. Consequently, $\text{Lem}((\Gamma, g), T) = \text{Lem}((\Gamma, g), T')$.*

2.2 Fragment Discovery via POG Decomposition

By Theorem 1, an extractable proof fragment $((\Gamma_F, g_F), \Upsilon)$ of a proof $((\Gamma, g), T)$ is determined by two subgraphs of G_T : (i) a subgraph H whose linearizations form the equivalence class of Υ , and (ii) a subgraph H_{pre} whose linearizations, applied to (Γ, g) , yield (Γ_F, g_F) . This motivates the two phases below.

Phase A: POG Decomposition. Given $G_T = (V, E)$ constructed for a proof tree T , generate all admissible decompositions $(H_{\text{pre}}, H, H_{\text{post}})$ of G_T , where H determines the fragment's proof tree and H_{pre} determines its input obligation. Admissibility is defined in §2.3.

Phase B: Obligation-annotated Proof Tree Construction Given an admissible decomposition $(H_{\text{pre}}, H, H_{\text{post}})$ of G_T and the root obligation of the proof, compute (i) the input obligation (Γ_F, g_F) of the fragment, and (ii) an obligation-annotated proof tree NDF obtained by linearizing H , in which every node is labeled with the obligation at which its tactic is executed. The annotation is required because the following phases consume obligation-annotated trees. For fragments drawn from reorderings of T , these obligations are not recorded in the original proof run and must be reconstructed.

2.3 Phase A: POG Decomposition

Let $G_T = (V, E)$. We start by a partition of V into three disjoint sets $V_H, V_{\text{pre}}, V_{\text{post}}$; the *decomposition* of G_T is consisted of $(H, H_{\text{pre}}, H_{\text{post}})$, the induced subgraphs of G_T on these vertex sets. A decomposition is *admissible* if it satisfies the conditions below.

Lemma vertices. The vertex set V_H must satisfy:

1. *Branch closure.* For any $a_i, a_j \in V_H$ with branch paths $B(a_i)$ and $B(a_j)$ and $B(a_i) \neq B(a_j)$, let $(b_1, \dots, b_n)$ be the longest common prefix of $B(a_i)$ and $B(a_j)$, then a must be in V_H where $B(a) = (b_1, \dots, b_n^*)$.
2. *Convexity.* For any $a_i, a_j \in V_H$ and any directed path $a_i \rightarrow \dots \rightarrow a_j$ in G_T , every node along the path lies in V_H .

Pre-set vertices. The vertex set $V_{\text{pre}} \subseteq V \setminus V_H$ must satisfy:

3. *Dependency closure.* For every edge $(a, a') \in E$ with $a' \in V_H$ and $a \notin V_H$, we have $a \in V_{\text{pre}}$.
4. *Pre-set consistency.* For every edge $(a, a') \in E$ with $a, a' \in V \setminus V_H$, if $a' \in V_{\text{pre}}$ then $a \in V_{\text{pre}}$.

Post-set vertices. $V_{\text{post}} = V \setminus (V_H \cup V_{\text{pre}})$.

2.3.1 Phase B: Obligation-annotated Proof Tree Construction

A *template* is a term S drawn from the grammar

$$S ::= \square_i \mid f(S_1, \dots, S_n),$$

where $\square_i$ ranges over numbered holes and f ranges over term constructors. Given a substitution σ mapping holes to terms, $S[\sigma]$ denotes the result of replacing each $\square_i$ with $\sigma(\square_i)$.

Definition 4 (Tactic Action). *A tactic action is one of:*

1. **Modify**($p, S_{\text{old}}, S_{\text{new}}$), where p identifies a goal or hypothesis and $S_{\text{old}}, S_{\text{new}}$ are templates. The action p matching S_{old} under substitution σ with $S_{\text{new}}[\sigma]$.
2. **Introduce**(c), where c is a proposition. The action adds a fresh hypothesis c to the context.

Definition 5 (Tactic Effect). *Let a be a tactic application at obligation (Γ, g) producing residual obligations $(\Gamma_1, g_1), \dots, (\Gamma_m, g_m)$. The effect of a at (Γ, g) is*

$$E(a, \Gamma, g) = (\vec{\alpha}_1, \dots, \vec{\alpha}_m),$$

an m -tuple of action sequences, one per residual obligation. Each $\vec{\alpha}_j = [\alpha_j^1, \dots, \alpha_j^n]$ describes the modifications that a performs on the proof obligation leading to (Γ_j, g_j) : applying $\alpha_1^j, \dots, \alpha_{n_j}^j$ in sequence to (Γ, g) yields (Γ_j, g_j) .

Definition 6 (Action Application). *Let (Γ, g) be a proof obligation and α a tactic action. Define $\text{Apply}(\alpha, \Gamma, g)$ as follows.*

- If $\alpha = \text{Modify}(p, S_{\text{old}}, S_{\text{new}})$: Let $\sigma = \{\Box_1 \mapsto s_1, \dots, \Box_k \mapsto s_k\}$ be the unique substitution such that $S_{\text{old}}[\sigma] = p$. Then:

$$\text{Apply}(\alpha, \Gamma, g) = \begin{cases} (\Gamma, S_{\text{new}}[\sigma]) & \text{if } p = g, \\ ((\Gamma \setminus \{p\}) \cup \{S_{\text{new}}[\sigma]\}, g) & \text{if } p \in \Gamma. \end{cases}$$

- If $\alpha = \text{Introduce}(c)$:

$$\text{Apply}(\alpha, \Gamma, g) = (\Gamma \cup \{c\}, g).$$

TODO Given an admissible decomposition $(H_{\text{pre}}, H, H_{\text{post}})$:

1. Linearize H_{pre} respecting G_T ; compose effects from the root obligation to obtain (Γ_F, g_F) .
2. Linearize H respecting G_T ; compose effects from (Γ_F, g_F) to obtain obligations at every node of H and at every boundary point.

3 Support Minimization

3.1 Support Calculus

The second component of the framework is a backward support analysis. Its goal is to compute, compositionally, the support required at the root of a selected partial proof tree. This analysis is parameterized by information recorded from the execution of the original proof script.

Consider an occurrence of an atomic tactic a at some obligation (Γ, g) , and suppose that

$$\llbracket a \rrbracket(\Gamma, g) = \{(\Gamma_1, g_1), \dots, (\Gamma_m, g_m)\}.$$

Definition 7 (Tactic summary). *A summary of this tactic occurrence is*

$$\Sigma(a, \Gamma, g) \triangleq (U, \pi_1, \dots, \pi_m),$$

where:

$\overline{\Gamma; g \vdash \bullet_\ell[(\Gamma_\ell, g_\ell)] : \emptyset}$	(SUPP-HOLE)
$\frac{\llbracket a \rrbracket(\Gamma, g) = \emptyset \quad \Sigma(a, \Gamma, g) = (U)}{\Gamma; g \vdash a : U}$	(SUPP-LEAF)
$\frac{\begin{array}{c} \llbracket a \rrbracket(\Gamma, g) = \{(\Gamma_1, g_1), \dots, (\Gamma_m, g_m)\} \quad \Sigma(a, \Gamma, g) = (U, \pi_1, \dots, \pi_m) \\ \Gamma_i; g_i \vdash \hat{P}_i : A_i \text{ for all } 1 \leq i \leq m \end{array}}{\Gamma; g \vdash a\langle \hat{P}_1, \dots, \hat{P}_m \rangle : U \cup \bigcup_{i=1}^m \bigcup_{h \in A_i} \pi_i(h)}$	(SUPP-COMP)

Figure 1: Support calculus

- $U \subseteq \Gamma$ is the set of hypotheses in the current context that are directly used by the tactic itself; and
- for each branch $1 \leq i \leq m$,

$$\pi_i : \Gamma_i \rightarrow \mathcal{P}(\Gamma)$$

is a dependency map such that, for each residual hypothesis $h \in \Gamma_i$, the set $\pi_i(h)$ consists of the hypotheses in the parent context Γ on which h depends.

The summary $\Sigma(a, \Gamma, g)$ records exactly the local information needed for backward support propagation through the tactic occurrence a at obligation (Γ, g) . The set U captures the hypotheses consumed directly by the tactic itself, while the maps π_i describe how support requirements arising in the residual obligations should be translated back to support requirements in the parent context. Accordingly, if branch i must preserve a support set $B_i \subseteq \Gamma_i$, then the support required at the input obligation (Γ, g) is

$$U \cup \bigcup_{i=1}^m \bigcup_{h \in B_i} \pi_i(h).$$

This is exactly the support that must be available at the current node in order for the tactic a to replay successfully and still leave available, at each residual obligation (Γ_i, g_i) , the support required by the continuation below branch i .

We define *support-annotated partial proof trees* by the grammar

$$\hat{P} ::= \bullet_\ell[(\Gamma_\ell, g_\ell)] \mid a \mid a\langle \hat{P}_1, \dots, \hat{P}_m \rangle,$$

The intended meaning of $\bullet_\ell[(\Gamma_\ell, g_\ell)]$ is that the selected fragment stops at a hole identified by ℓ , exposing the residual obligation (Γ_ℓ, g_ℓ) from the original proof. The residual obligation is retained at each hole so that it can be referenced when constructing the final lemma statement.

The support calculus computes the support required at the root of such a support-annotated partial proof tree. Its judgment has the form

$$\Gamma; g \vdash \hat{P} : A,$$

where (Γ, g) is the root obligation of the partial proof tree and $A \subseteq \Gamma$ is the support required at the root in order to replay the selected fragment while preserving the support annotations recorded at its holes. Its intended meaning is as follows: if the partial proof tree $\hat{P}$ is replayed from any root context $\Gamma' \subseteq \Gamma$ such that $A \subseteq \Gamma'$, then replay succeeds along the selected fragment and every hole $\bullet_\ell[(\Gamma_\ell, g_\ell)]$ is reached.

The calculus is presented in Figure 1. The SUPP-HOLE rule states that a hole contributes no support requirements, since the support required for a continuation tree is irrelevant to the support required for the chosen tree segment. The SUPP-LEAF rule handles a selected tactic leaf whose replay ends immediately after executing that tactic; the required support is exactly the set of hypotheses used directly by that tactic occurrence. The SUPP-COMP rule uses the summary $\Sigma(a, \Gamma, g)$ of the tactic occurrence at the current node to pull the support requirements of the children back to the parent context.